PROGRAMACIÓN III

INFORME TÉCNICO

AVANCE DEL PROYECTO – PROGRESO 1
Análisis, diseño y selección de la solución

TEMA: Sistema de recomendación y matching para la adopción de mascotas en un refugio

ESTUDIANTES: Zaid Chamorro, Esteban Chapaca, Mahe Paredes

QUITO – ECUADOR

1. Formulación del problema

Contexto y necesidad

Los refugios de animales de compañía suelen funcionar con poco personal, voluntarios rotativos y espacio limitado. En Quito, como en otras ciudades del país, el abandono de perros y gatos es visible, y buena parte de la atención recae en fundaciones y refugios que necesitan que los animales salgan en adopción para poder recibir a otros. El proceso de adopción, sin embargo, sigue siendo mayoritariamente manual: alguien del refugio conversa con el interesado y le muestra los animales que, a su criterio, encajan con su estilo de vida. Ese emparejamiento "a ojo" depende de la experiencia de quien atiende, es lento con decenas o cientos de animales y no deja registro de por qué se recomendó a uno y no a otro.

Las consecuencias de un mal emparejamiento están bien documentadas. Hawes et al. (2020) analizaron las devoluciones de animales adoptados en un refugio de Estados Unidos y encontraron que, en perros, el motivo más frecuente fueron los problemas de comportamiento relacionados con agresividad (38,2 %), mientras que en gatos predominaron razones personales del adoptante. Powell et al. (2021), con casi 24 000 registros, identificaron los problemas de conducta y la incompatibilidad con otras mascotas del hogar como las causas más comunes de devolución, y mostraron que los perros adultos tienen bastante más probabilidad de ser devueltos que los cachorros. Muchas devoluciones, entonces, no se deben a "malos animales", sino a una incompatibilidad que una evaluación sistemática podría haber anticipado.

El otro lado del problema es el tiempo de permanencia. Brown et al. (2013) encontraron que la estadía en refugio aumenta con la edad del perro: los cachorros salen rápido y los adultos y seniors se quedan más tiempo. Bradley y Rajendran (2021) señalan que la edad, el tamaño y el color están entre los factores que más influyen en la duración de la estadía, y que estadías muy largas se asocian con hacinamiento y, en algunos refugios, con eutanasia.

Por eso necesitamos un sistema que haga dos cosas a la vez: recomendar a cada adoptante los animales con los que tiene mayor afinidad real (para reducir devoluciones) y dar visibilidad a los animales que llevan más tiempo esperando (para reducir estadías largas), sin que lo segundo sacrifique lo primero.

Actores involucrados

- **Personal y voluntarios del refugio:** registran animales, evalúan su temperamento, revisan solicitudes y hacen las visitas domiciliarias y de seguimiento.
- **Coordinador/administrador del refugio:** define reglas y pesos del matching, consulta reportes y toma decisiones de capacidad.
- **Adoptantes:** registran su perfil, su vivienda y sus preferencias, y reciben recomendaciones.
- **Veterinario (actor secundario):** aporta el estado de salud y las necesidades especiales de cada animal.

Causas y consecuencias

Entre las **causas** identificamos: (1) matching manual basado en la intuición de quien atiende; (2) información del animal dispersa o incompleta, especialmente sobre temperamento; (3) preferencias del adoptante que no se contrastan con su realidad (vivienda, tiempo disponible, niños u otras mascotas); (4) ausencia de un mecanismo que priorice a los animales con estadías largas; y (5) visitas al hogar sin planificación, que consumen tiempo del personal. Las **consecuencias** son devoluciones por incompatibilidad, estrés para el animal devuelto, mayor ocupación del refugio, animales adultos y seniors que envejecen sin ser adoptados y ausencia de datos para mejorar el proceso.

Datos y procesos principales

Los datos centrales son la ficha del animal (especie, edad, sexo, tamaño, estado de salud, necesidades especiales, temperamento y fecha de ingreso), el perfil del adoptante (edad, sector donde vive, vivienda, patio, horas fuera de casa, niños, otras mascotas, experiencia previa), sus preferencias con pesos por criterio, las solicitudes con su estado, las visitas al hogar con su resultado y el historial de devoluciones con su motivo. El proceso que modelamos es: ingreso y evaluación del animal → registro y precalificación del adoptante → recomendación de los N animales más compatibles → solicitud → visita domiciliaria → aprobación o rechazo → visitas de seguimiento → eventual devolución, que vuelve a alimentar la priorización y los reportes.

Variables relevantes y restricciones

Variables: edad del animal (meses), tamaño, niveles de energía y ansiedad (1–5), sociabilidad con niños, perros y gatos, días en refugio, número de devoluciones previas, tipo de vivienda, disponibilidad de patio, horas fuera de casa, presencia de niños u otras mascotas, experiencia previa, pesos de preferencia por criterio, sector de residencia y tiempos de traslado entre sectores. **Salidas:** precalificación del adoptante, puntaje de afinidad (0–1), índice de prioridad (0–1), puntaje final con su desglose y orden de las visitas del día.

Restricciones del proyecto: implementación en Java (JDK 24) en IntelliJ IDEA, usando solo la biblioteca estándar de Java y las estructuras de datos y algoritmos que se estudian en la asignatura (no solo CRUD); persistencia en archivos CSV; interfaz de consola; un equipo de tres integrantes y el calendario del semestre. Usaremos datos sintéticos realistas, porque no contamos con datos de un refugio real. **Restricciones del dominio:** un animal solo puede tener una solicitud aprobada a la vez; la priorización nunca debe recomendar un animal con afinidad baja; y los datos personales de los adoptantes deben tratarse según la Ley Orgánica de Protección de Datos Personales (Asamblea Nacional del Ecuador, 2021).

Relación con factores globales

El problema tiene una dimensión **social** (bienestar animal y tenencia responsable), **económica** (cada día de estadía cuesta alimento, atención veterinaria y espacio a refugios que suelen vivir de donaciones) y de **salud pública urbana**, porque la capacidad de los refugios condiciona cuántos animales en situación de calle pueden ser recibidos. Los demás factores no tienen relación directa con el alcance.

2. Alcance y módulos funcionales

El sistema se organiza en seis módulos. Los módulos 1 y 2 alimentan al motor de recomendación (módulo 3); el módulo 4 aporta el índice de prioridad que el motor combina con la afinidad; el módulo 5 registra el resultado de cada solicitud, y el módulo 6 programa y ordena las visitas al hogar que el refugio necesita para aprobar una adopción y acompañarla después. A los cinco módulos de la propuesta inicial les sumamos el sexto porque la visita domiciliaria es un paso habitual del proceso de adopción y planificarla es un problema de rutas. La asignación de responsables es preliminar y puede ajustarse durante el desarrollo.

Módulo 1. Registro de animales

Elemento	Descripción
Propósito	Mantener la ficha completa y estandarizada de cada animal, incluida una evaluación de temperamento en escala común, para que el matching use datos comparables.
Actor principal	Personal del refugio (con aporte del veterinario).

Elemento	Descripción
Datos	Animal (Perro/Gato), Temperamento, EstadoSalud, fecha de ingreso, estado (DISPONIBLE, EN_PROCESO, ADOPTADO, DEVUELTO), número de devoluciones. Se guardan en una lista dinámica (`ArrayList<Animal>`) ordenada por id; las ediciones recientes de las fichas, en una pila de cambios.
Operaciones	1) Registrar animal con validación de campos; se agrega al final de la lista con un id correlativo (A0001, A0002, ...), así la lista queda ordenada por id. 2) Editar la ficha (temperamento, salud); cada edición apila el valor anterior. 3) Deshacer la última edición desapilando el cambio (pila LIFO), O(1). 4) Buscar animal por id con búsqueda binaria, O(log n). 5) Filtrar animales por especie, rango de edad, tamaño y estado con búsqueda secuencial, O(n). 6) Cambiar estado del animal según el ciclo de adopción. 7) Calcular días en refugio y marcar seniors (perro ≥ 7 años, gato ≥ 10 años).
Reglas de negocio	Un animal sin evaluación de temperamento no aparece en recomendaciones. Solo los animales DISPONIBLES o DEVUELTOS (tras reevaluación) son recomendables. La capacidad máxima del refugio no puede superarse. La pila guarda las últimas 20 ediciones; los cambios de estado de adopción no se deshacen desde aquí, porque los controla el Módulo 5.
Relación	Provee candidatos al Módulo 3; provee días en refugio y devoluciones al Módulo 4; su estado lo actualiza el Módulo 5.
Responsable	Esteban Chapaca

Módulo 2. Registro de adoptantes y preferencias

Elemento	Descripción
Propósito	Capturar el perfil real del hogar del adoptante y sus preferencias, con pesos ajustables por criterio, para contrastar “lo que quiere” con “lo que su hogar permite”.
Actor principal	Adoptante (registro) y personal del refugio (validación).
Datos	Adoptante (cédula, edad, contacto, sector, vivienda, permiso para mascotas, patio, horas fuera de casa, niños, otras mascotas, experiencia, aceptación de visitas), Preferencia (especies, rango de edad, tamaños, energía deseada, acepta necesidades especiales, pesos por criterio). Los adoptantes se guardan en un árbol binario de búsqueda (ABB) ordenado por cédula y los pesos en un arreglo de 6 posiciones (`double[]`), uno por criterio.
Operaciones	1) Registrar adoptante con validación de cédula ecuatoriana (dígito verificador) e insertarlo de forma recursiva en el ABB, O(log n) en promedio. 2) Precalificar al adoptante con un árbol de decisión (mayor de edad, acepta visitas, vivienda que permite mascotas, horas fuera de casa): APTO, APTO CON CONDICIONES o NO APTO, O(h). 3) Definir y actualizar preferencias y pesos (normalizados para sumar 1). 4) Detectar inconsistencias perfil–preferencia (p. ej., perro grande sin patio) y advertirlas. 5) Buscar adoptante por cédula en el ABB, O(log n) en promedio. 6) Listar adoptantes ordenados por cédula con un recorrido inorden, O(n).
Reglas de negocio	Un adoptante NO APTO no puede solicitar ni recibe recomendaciones. Un adoptante tiene una única Preferencia vigente y solo una solicitud pendiente a la vez. Si tiene niños, se excluyen animales no sociables con niños (filtro duro).
Relación	Provee perfil, preferencias y precalificación al Módulo 3 y el sector del adoptante al Módulo 6; sus solicitudes quedan en el Módulo 5.
Responsable	Esteban Chapaca

Módulo 3. Motor de recomendación (matching por afinidad)

Elemento	Descripción
Propósito	Generar para cada adoptante una lista ordenada de los N animales más compatibles, con un puntaje explicable por criterio.
Actor principal	Adoptante (consulta) y personal del refugio (revisión).
Datos	Animales recomendables (lista del Módulo 1), Adoptante, Preferencia, índice de prioridad (Módulo 4); produce objetos Compatibilidad, que implementan `Comparable`.
Operaciones	1) Aplicar filtros duros (especie, estado, niños, necesidades especiales) recorriendo la lista de animales. 2) Calcular afinidad ponderada por criterio (edad, tamaño, energía, vivienda, convivencia, experiencia). 3) Combinar afinidad y prioridad en un puntaje final. 4) Seleccionar el top-N con una cola con prioridad (`PriorityQueue`, min-heap) de tamaño N. 5) Explicar la recomendación (desglose del puntaje por criterio). 6) Recomendar adoptantes compatibles para un animal concreto (matching inverso).
Reglas de negocio	El bono de prioridad solo se aplica si la afinidad ≥ 0,50, para que la urgencia nunca genere un mal emparejamiento. Los adoptantes APTOS CON CONDICIONES solo reciben animales adultos (≥ 2 años) con energía ≤ 3. Empates: gana el animal con más días en refugio. N por defecto = 5.
Relación	Consume los Módulos 1, 2 (incluida la precalificación) y 4; la Compatibilidad elegida se guarda con la solicitud en el Módulo 5.
Responsable	Zaid Chamorro

Módulo 4. Priorización de animales por tiempo en refugio

Elemento	Descripción
Propósito	Dar visibilidad a los animales con estadías largas, seniors o devueltos, mediante un índice de prioridad y una cola de prioridad.
Actor principal	Coordinador del refugio.
Datos	Días en refugio, edad (senior), número de devoluciones; estructura `PriorityQueue<Animal>` (max-heap por índice).
Operaciones	1) Calcular el índice de prioridad de cada animal. 2) Reconstruir la cola insertando los animales adoptables, O(n log n), al inicio de cada jornada o tras cambios masivos. 3) Consultar los k animales más prioritarios en O(k log n). 4) Generar alertas para animales que superan un umbral de días (p. ej., 90). 5) Actualizar la prioridad de un animal al registrarse una devolución.
Reglas de negocio	Índice = 0,7·min(1, días/180) + 0,2·(senior) + 0,1·min(1, devoluciones). Un animal ADOPTADO sale de la cola. Los pesos del índice son configurables por el coordinador.
Relación	Lee datos del Módulo 1 y del Módulo 5 (devoluciones); entrega el índice al Módulo 3.
Responsable	Zaid Chamorro

Módulo 5. Historial de adopciones y reportes

Elemento	Descripción
Propósito	Registrar el ciclo completo de cada solicitud y generar indicadores que permitan evaluar si el matching está reduciendo devoluciones y estadías.
Actor principal	Personal del refugio (registro) y coordinador (reportes).

Elemento	Descripción
Datos	SolicitudAdopcion (fechas, estado, Compatibilidad registrada, motivo de devolución); `ArrayList` cronológico de solicitudes y una cola FIFO (Queue) de solicitudes pendientes.
Operaciones	1) Crear solicitudes: se agregan al historial y al final de la cola de pendientes. 2) Atender la siguiente solicitud pendiente en orden de llegada (FIFO) para aprobarla —si su visita previa fue favorable— o rechazarla, con el cambio de estado del animal. 3) Registrar devoluciones con motivo y fecha. 4) Calcular tasa de devolución global y por rango de puntaje de compatibilidad. 5) Calcular estadía promedio por especie y por grupo de edad. 6) Ranking de motivos de devolución: ordenarlos con mergesort (recursivo), contar las repeticiones consecutivas y ordenar por frecuencia, O(m log m). 7) Buscar solicitudes por rango de fechas con búsqueda binaria sobre la lista cronológica. 8) Exportar reportes a CSV.
Reglas de negocio	Las solicitudes pendientes se revisan en orden de llegada y solo se aprueban con una visita previa favorable (Módulo 6). Una solicitud aprobada cambia el animal a ADOPTADO y rechaza automáticamente las demás pendientes del mismo animal. Una devolución solo puede registrarse sobre una solicitud APROBADA.
Relación	Recibe solicitudes del Módulo 3; pide al Módulo 6 la visita previa y los seguimientos; actualiza el estado en el Módulo 1 y las devoluciones para el Módulo 4.
Responsable	Mahe Paredes

Módulo 6. Visitas domiciliarias y seguimiento posadopción

Elemento	Descripción
Propósito	Programar la visita al hogar que exige la aprobación de una adopción y las visitas de seguimiento posteriores, y ordenar las visitas de cada día para reducir el tiempo de traslado del personal.
Actor principal	Personal del refugio (visitas) y coordinador (planificación).
Datos	Mapa de sectores de la ciudad como grafo ponderado no dirigido (nodos = sectores; aristas = minutos de traslado) guardado en una lista de adyacencia; sector del adoptante; agenda de visitas en una `PriorityQueue<Visita>` ordenada por fecha; matriz (arreglo bidimensional) de tiempos mínimos entre los puntos del día.
Operaciones	1) Cargar el mapa de sectores y verificar que sea conexo con un recorrido en anchura (BFS, usa una cola), O(V + E). 2) Calcular el tiempo mínimo desde el refugio hasta cada sector con el algoritmo de Dijkstra, usando una cola con prioridad, O((V + E) log V). 3) Programar la visita previa al crear una solicitud y las visitas de seguimiento a los 30 y 90 días de la adopción. 4) Obtener las visitas de un día sacándolas de la agenda en orden de fecha, O(k log n). 5) Ordenar la ruta del día: con Dijkstra desde cada punto se arma la matriz de tiempos mínimos entre los k lugares a visitar; sobre ella se construye el árbol de expansión mínima (Prim, $O(k^2)$) y se recorre en preorden desde el refugio. La ruta resultante no supera el doble de la óptima (Cormen et al., 2022). 6) Registrar el resultado de cada visita (favorable, con observaciones o desfavorable).
Reglas de negocio	Una solicitud solo puede aprobarse si su visita previa fue favorable. Cada día se programa un máximo configurable de visitas (p. ej., 6). Si una visita de seguimiento resulta desfavorable, se alerta al coordinador por posible devolución.
Relación	Recibe solicitudes del Módulo 5 y el sector del adoptante del Módulo 2; devuelve al Módulo 5 el resultado de la visita previa.
Responsable	Mahe Paredes

Nota: las operaciones de registro (CRUD) existen en los módulos 1, 2, 5 y 6, pero el núcleo del proyecto son las operaciones algorítmicas: búsqueda secuencial y binaria, pila para deshacer, árbol binario de

búsqueda y árbol de decisión, scoring ponderado, colas y colas con prioridad, ordenamiento con mergesort y algoritmos sobre grafos (recorrido en anchura, Dijkstra y árbol de expansión mínima).

3. Modelo orientado a objetos y diagrama UML preliminar

El diagrama se elaboró en PlantUML (los archivos fuente están en `docs/uml/` del repositorio) y fue revisado por el equipo contra los módulos definidos. Por su tamaño lo dividimos en dos vistas: la Figura 1 muestra el modelo de dominio —las seis clases propuestas (Animal, Adoptante, Preferencia, SolicitudAdopcion, Refugio y Compatibilidad) con sus subclases, enumeraciones y relaciones— y la Figura 2, los servicios y las estructuras de datos propias (árboles, grafo y colas) que implementan los módulos algorítmicos. En la Figura 2, las clases que aparecen solo con su nombre están detalladas en la Figura 1. Las versiones en alta resolución se incluyen en el repositorio.

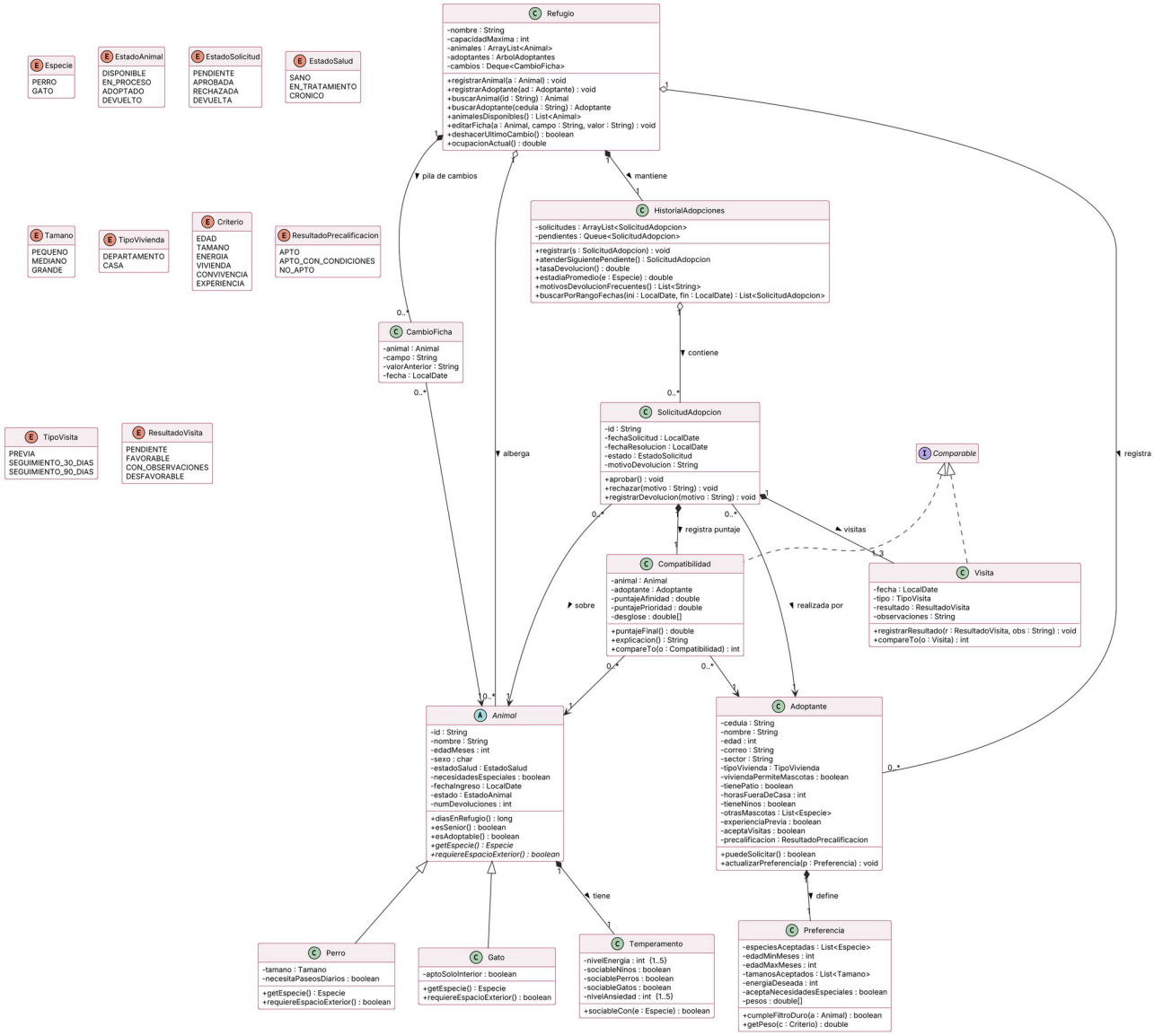


Figura 1. Modelo de dominio (diagrama de clases UML preliminar).

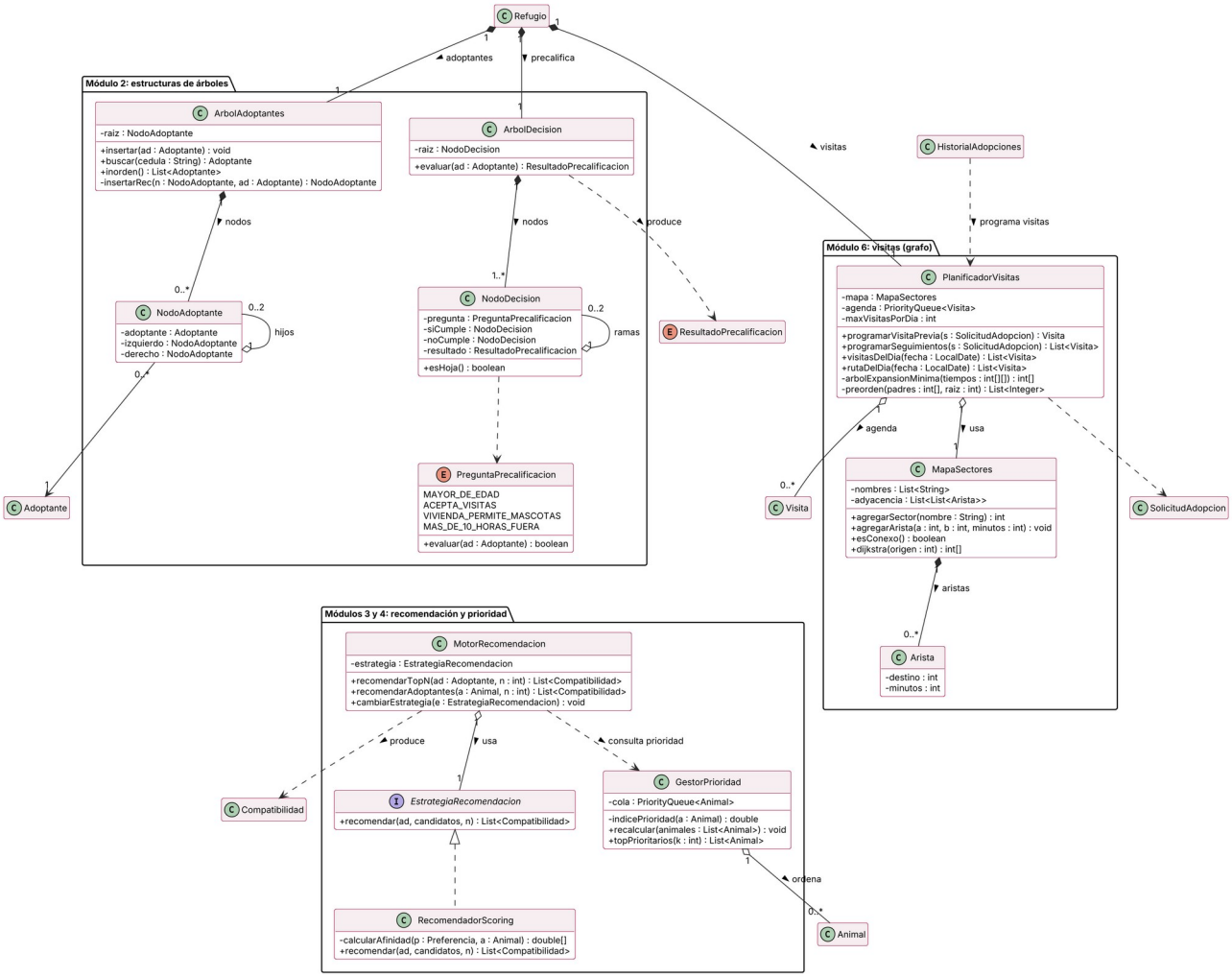


Figura 2. Servicios y estructuras de datos (diagrama de clases UML preliminar).

Decisiones de diseño

- **Herencia (Animal → Perro, Gato):** se justifica porque las reglas de compatibilidad cambian según la especie: el tamaño y la necesidad de paseos solo aplican a perros, y la condición de "apto solo interior" a gatos. `Animal` es abstracta y declara `requiereEspacioExterior()`, que cada subclase implementa. Así el motor no necesita condicionales por especie.
- **Composición Animal ◆— Temperamento y Adoptante ◆— Preferencia:** el temperamento no existe sin su animal y la preferencia no existe sin su adoptante; se crean y destruyen junto con ellos.
- **Agregación Refugio ◇— Animal / Adoptante:** el refugio los administra, pero un animal adoptado o un adoptante siguen existiendo como registros fuera de la colección activa.
- **Composición Refugio ◆— HistorialAdopciones y SolicitudAdopcion ◆— Compatibilidad / Visita:** el historial pertenece al refugio; cada solicitud guarda una "fotografía" del puntaje que tenía cuando se creó, lo que permite medir después si los puntajes altos se devuelven menos, y sus visitas (la previa y las de seguimiento) no existen sin ella.
- **Estructuras estáticas y dinámicas:** los pesos de los criterios se guardan en un arreglo de 6 posiciones porque los criterios son fijos y se accede a cada uno por su posición (`Criterio.ordinal()`); lo mismo ocurre con la matriz de tiempos entre los puntos de una ruta, cuyo tamaño se conoce al armarla. En cambio, la cantidad de animales, adoptantes, solicitudes y visitas cambia todo el tiempo, por eso usan estructuras dinámicas: listas, pilas, colas y árboles.

- **Lista ordenada para animales y ABB para adoptantes:** los animales reciben ids correlativos, así que la lista queda ordenada sola y permite búsqueda binaria; un ABB con esos ids se degeneraría en una lista. Las cédulas, en cambio, llegan en orden aleatorio, por lo que el ABB se mantiene equilibrado en promedio y permite insertar y buscar en O(log n) sin desplazar elementos.
- **Árbol de decisión para la precalificación:** las reglas de admisión del refugio se representan como un árbol en el que cada nodo interno es una pregunta de sí o no y cada hoja, un resultado. Cambiar o agregar una regla es modificar un nodo, sin reescribir una cadena de condicionales.
- **Grafo con lista de adyacencia:** el mapa de sectores es disperso (cada sector se conecta solo con sus vecinos), así que la lista de adyacencia ocupa $O(V + E)$, frente a $O(V^2)$ de una matriz de adyacencia.
- **Interfaz EstrategiaRecomendacion (patrón Strategy):** el motor depende de una interfaz, no de una implementación. Implementamos la alternativa A (`RecomendadorScoring`) y, si en el futuro se necesitara el ranking completo de candidatos, la alternativa B (árbol binario de búsqueda) podría añadirse como otra implementación sin tocar el resto del sistema.

Correspondencia entre módulos y clases

Módulo	Clases principales	Estructura de datos clave
1. Registro de animales	Animal, Perro, Gato, Temperamento, Refugio, CambioFicha	`ArrayList` ordenada por id + búsqueda binaria; pila (Deque) de cambios
2. Adoptantes y preferencias	Adoptante, Preferencia, ArbolAdoptantes, NodoAdoptante, ArbolDecision, NodoDecision	ABB por cédula + recorrido inorden; árbol de decisión; arreglo `double[]` de pesos
3. Motor de recomendación	MotorRecomendacion, EstrategiaRecomendacion, RecomendadorScoring, Compatibilidad	`PriorityQueue` (min-heap top-N) + `Comparable`
4. Priorización	GestorPrioridad	`PriorityQueue` (max-heap)
5. Historial y reportes	HistorialAdopciones, SolicitudAdopcion	`ArrayList` cronológica + búsqueda binaria, `Queue` (FIFO), mergesort
6. Visitas	MapaSectores, Arista, PlanificadorVisitas, Visita	Grafo con lista de adyacencia; BFS, Dijkstra, Prim + preorden; `PriorityQueue` de visitas

4. Alternativas basadas en estructuras de datos y algoritmos

El componente más relevante de la solución es el motor de recomendación (módulo 3), porque concentra la operación algorítmica central. Planteamos dos alternativas técnicamente diferenciadas. En la notación, **n** es el número de animales recomendables, **f** el número de criterios (en nuestro caso f = 6) y **N** el tamaño de la lista recomendada ($N \ll n$).

Alt.	Estructura(s)	Algoritmo(s)	Operación que resuelve	Complejidad temporal	Complejidad espacial	Ventajas / limitaciones
A	Lista de animales (ArrayList) + arreglo de pesos + cola con prioridad (PriorityQueue, min-heap de tamaño N).	Recorrido secuencial con filtros duros + scoring ponderado (suma de pesos × similitud por criterio) + selección top-N con la cola con prioridad (`Comparable`).	Recomendar los N animales más afines a un adoptante, con desglose del puntaje.	O(n·f + n log N) ≈ **O(n·f)** por consulta, porque N es pequeño y constante.	**O(n)** para la lista; O(N) adicional para la cola.	**V:** admite filtros duros y pesos distintos por adoptante; cada puntaje es explicable; usa la `PriorityQueue` de Java, sin estructuras propias. **L:** recorre todos los candidatos en cada consulta; solo conserva los N mejores, así que un ranking completo exige recalcular.
B	Árbol binario de búsqueda (ABB) cuyos nodos guardan la Compatibilidad de cada candidato, ordenado por puntaje.	Recorrido secuencial con filtros duros + scoring ponderado + inserción recursiva en el ABB + recorrido inorden inverso (derecha–raíz–izquierda) que se detiene al obtener N.	Recomendar por afinidad y, además, obtener el ranking completo o los candidatos dentro de un rango de puntaje.	O(n·f + n log n) **promedio**; **O(n·f + n²) en el peor caso** (árbol degenerado).	**O(n)**: un nodo por candidato en cada consulta.	**V:** deja todos los candidatos ordenados (ranking completo, rangos, "siguientes N" sin recalcular); aplica recursividad y recorridos de árboles. **L:** se degenera en una lista cuando muchos puntajes son iguales o llegan ordenados; hay que implementar y probar el árbol a mano; construirlo cuesta más que la cola cuando solo se necesitan N.

Detalle de la alternativa A: filtrado + scoring + top-N

1. Filtrado (filtros duros). Se recorre la lista de animales (búsqueda secuencial, O(n)) y se descartan los no disponibles, los de una especie que el adoptante no acepta, los no sociables con niños si el hogar tiene niños y los que tienen necesidades especiales si el adoptante no las acepta.

2. Scoring ponderado. Para cada candidato se calcula una similitud $s_c \in [0, 1]$ por criterio y la afinidad como promedio ponderado: afinidad = $\Sigma\, w_c \cdot s_c$, con $\Sigma\, w_c = 1$. Por ejemplo, energía: $s = 1 - |e_{animal} - e_{deseada}| / 4$; edad: $s = 1$ dentro del rango preferido y decrece linealmente hasta 0 a 12 meses fuera de él; vivienda: se penaliza un perro que requiere espacio exterior en un hogar sin patio. Los pesos se guardan en un arreglo de 6 posiciones; por defecto son edad 0,20; tamaño 0,15; energía 0,20; vivienda 0,20; convivencia 0,15; experiencia 0,10, y cada adoptante puede ajustarlos.

3. Combinación con prioridad. Si afinidad ≥ 0,50, el puntaje final es 0,85 · afinidad + 0,15 · prioridad (índice del módulo 4). Si la afinidad es menor, el animal no recibe bono. Así, entre dos animales igual de compatibles, se favorece al que lleva más tiempo esperando, pero nunca se recomienda un mal emparejamiento por urgencia.

4. Selección top-N. Compatibilidad implementa la interfaz `Comparable`, así la `PriorityQueue` sabe qué candidato tiene menor puntaje. Se recorre la lista con una cola de tamaño máximo N ordenada de menor a mayor puntaje (min-heap): si la cola tiene menos de N elementos se inserta; si el candidato supera al mínimo, se extrae el mínimo y se inserta el candidato. Cada operación cuesta O(log N), por lo que la selección completa es O(n log N), mejor que ordenar todo en O(n log n) (Cormen et al., 2022).

Detalle de la alternativa B: árbol binario de búsqueda y recorrido inorden

Después de filtrar y calcular el puntaje (igual que en A), cada Compatibilidad se inserta de forma recursiva en un ABB usando el puntaje final como clave: los puntajes menores van al subárbol izquierdo y los mayores o iguales, al derecho. Luego, un recorrido inorden inverso (primero el

subárbol derecho, después la raíz y al final el izquierdo) visita los candidatos de mayor a menor puntaje y se detiene al llegar a N. Si los puntajes llegan en orden aleatorio, la altura del árbol es O(log n) en promedio y cada inserción cuesta O(log n); pero si llegan ordenados o hay muchos empates, los nodos terminan en una sola rama, la altura llega a O(n) y construir el árbol cuesta $O(n^2)$ (Cormen et al., 2022). En nuestro caso los empates son frecuentes, porque varios animales pueden obtener exactamente la misma afinidad.

5. Selección y fundamentación de la mejor alternativa

Comparamos ambas alternativas con los criterios de la consigna, considerando el tamaño realista de un refugio: entre decenas y unos pocos cientos de animales disponibles.

Criterio	Alternativa A (lista + scoring + PriorityQueue)	Alternativa B (ABB + recorrido inorden)
Adecuación al problema	Alta. La operación principal solo necesita los N mejores, y la cola con prioridad está hecha justamente para eso. Maneja filtros duros y pesos por adoptante, y explica el puntaje por criterio.	Media. Entrega un ranking completo que la operación principal no necesita, y los empates de puntaje (frecuentes) lo desbalancean.
Tiempo / complejidad	O(n·f + n log N). Con n = 500, f = 6 y N = 5 son unas 3 000 evaluaciones de criterio y unas 1 200 comparaciones en la cola.	O(n·f + n log n) en promedio: unas 3 000 evaluaciones y unas 4 500 comparaciones ($500 \cdot \log_2 500 \approx 4\,480$). En el peor caso, unas 125 000 comparaciones ($n^2/2$).
Uso de memoria	O(n) de la lista que ya existe, más O(N) para la cola.	O(n) adicional: un nodo nuevo por candidato en cada consulta.
Escalabilidad	Lineal en n; suficiente para varios miles de animales.	n log n en promedio, pero con riesgo de crecimiento cuadrático si el árbol se degenera.
Mantenibilidad	Alta. Usa la `PriorityQueue` de Java, ya probada; añadir un criterio es añadir un valor al enum `Criterio`, una posición al arreglo de pesos y su función de similitud.	Media. Hay que mantener código propio de nodos, inserción, recorrido y manejo de empates.
Dificultad de implementación	Razonable para el equipo y el plazo; usa la `PriorityQueue` de Java.	Media: requiere implementar el árbol con recursividad (inserción y recorrido inorden inverso) y resolver los empates.

Alternativa seleccionada: A (lista + filtrado + scoring ponderado + top-N con PriorityQueue). La operación central es recomendar los N animales más compatibles, y para eso no hace falta ordenar a todos los candidatos: la cola con prioridad de tamaño N lo resuelve en O(n log N), con menos memoria y sin riesgo de degenerarse, mientras que el ABB cuesta O(n log n) en promedio y puede llegar a $O(n^2)$ por los empates de puntaje. Además, A deja un código más simple de probar.

Esto no significa descartar los árboles: el ABB sí lo usamos donde encaja mejor, en el registro de adoptantes por cédula (módulo 2), porque las cédulas llegan en orden aleatorio y el árbol se mantiene equilibrado en promedio. Si en el futuro se necesitara el ranking completo de candidatos, la interfaz `EstrategiaRecomendacion` permite añadir la alternativa B como otra implementación, sin tocar el resto del sistema.

Estructuras y algoritmos de la asignatura en el proyecto

Además del motor de recomendación, el resto del sistema aplica los demás temas de la asignatura donde resuelven un problema concreto del refugio. La tabla resume dónde se usa cada uno; el único tema que no aplicamos son los árboles de juego, porque el problema no tiene dos jugadores que alternen jugadas.

Tema de la asignatura	Uso en el proyecto	Complejidad
Arreglos (estructura estática)	Pesos de los 6 criterios; matriz de tiempos entre los puntos de la ruta del día.	O(1) por acceso
Listas	Animales ordenados por id; historial cronológico de solicitudes.	O(1) agregar al final
Pilas	Deshacer la última edición de una ficha (módulo 1).	O(1) apilar y desapilar
Colas	Solicitudes pendientes en orden de llegada (módulo 5); recorrido en anchura del mapa (módulo 6).	O(1) encolar y desencolar
Colas con prioridad	Top-N de recomendaciones (módulo 3), priorización de animales (módulo 4), agenda de visitas y Dijkstra (módulo 6).	O(log n) insertar y extraer
Métodos de búsqueda	Búsqueda secuencial para filtrar candidatos; búsqueda binaria por id y por rango de fechas.	O(n) y O(log n)
Recursividad	Mergesort, inserción y búsqueda en el ABB, evaluación del árbol de decisión y recorrido en preorden.	—
Métodos de ordenamiento	Mergesort para el ranking de motivos de devolución y los reportes.	O(n log n)
Árboles binarios de búsqueda y recorridos	Adoptantes por cédula y listado en inorden (módulo 2).	O(log n) promedio; O(n) el recorrido
Árboles de decisión	Precalificación del adoptante (módulo 2).	O(h), h = altura
Grafos y ruta más corta	Mapa de sectores con lista de adyacencia; tiempos mínimos con Dijkstra y conectividad con BFS (módulo 6).	O((V + E) log V) y O(V + E)
Árboles de expansión	Orden de las visitas del día: árbol de expansión mínima con Prim y recorrido en preorden (módulo 6).	O(k^2)

Las complejidades corresponden a las implementaciones estándar de cada estructura y algoritmo (Cormen et al., 2022; Oracle, s. f.). Compatibilidad y Visita implementan `Comparable` de forma consistente para que las colas con prioridad las ordenen correctamente (Bloch, 2018).

6. Plan preliminar de implementación

Asignación de módulos y componentes

Organizamos la implementación según el calendario del sílabo: en el Progreso 2 construimos lo que se apoya en pilas, colas, colas con prioridad, listas, búsqueda y ordenamiento, y en el Progreso 3, lo que usa árboles y grafos.

Integrante	Módulos / componentes	Progreso 2	Progreso 3
Zaid Chamorro (@zaid290)	Módulo 3 (MotorRecomendacion, RecomendadorScoring, Compatibilidad), Módulo 4 (GestorPrioridad) e interfaz de consola.	Filtrado, scoring ponderado, top-N con PriorityQueue, explicación del puntaje, cola de prioridad de animales y menú de consola.	Integración con la precalificación del Módulo 2 y ajuste de pesos con casos de prueba.
Esteban Chapaca (@Esteban201020)	Módulo 1 (Animal, Perro, Gato, Temperamento, Refugio, CambioFicha), Módulo 2 (Adoptante, Preferencia, ArbolAdoptantes, ArbolDecision) y persistencia CSV.	Registro, búsqueda binaria y filtrado de animales; pila de deshacer; registro de adoptantes y validación de cédula; lectura y escritura de CSV.	ABB de adoptantes (inserción, búsqueda, inorden) y árbol de decisión de precalificación.

Integrante	Módulos / componentes	Progreso 2	Progreso 3
Mahe Paredes (@MaheP-06)	Módulo 5 (HistorialAdopciones, SolicitudAdopcion) y Módulo 6 (MapaSectores, PlanificadorVisitas, Visita).	Cola FIFO de solicitudes, mergesort, búsqueda binaria por fechas y reportes.	Mapa de sectores (grafo), BFS, Dijkstra, Prim, ruta de visitas y aprobación con visita previa favorable.
Todo el equipo	Integración, revisión de código (pull requests), datos sintéticos y documentación.	Generador de datos de prueba (≈ 300 animales, 100 adoptantes).	Pruebas integrales, informe final y presentación.

Estructura inicial del repositorio GitHub

```
refugio-matching/
├── README.md                  # descripción, cómo abrir y ejecutar, integrantes
├── refugio-matching.iml       # proyecto de IntelliJ IDEA (JDK 24)
├── .idea/                     # configuración compartida del proyecto
├── .gitignore
├── docs/
│   ├── informe-progreso1.pdf
│   ├── presentacion-progreso1.pdf
│   └── uml/                    # diagramas de clases (PlantUML)
├── data/
│   ├── animales.csv            # datos sintéticos de prueba
│   ├── adoptantes.csv
│   └── sectores.csv            # mapa de sectores (grafo)
├── src/ec/udla/refugio/
│   ├── App.java
│   ├── modelo/                 # Animal, Adoptante, SolicitudAdopcion, Visita,
│   │                           # CambioFicha y demás clases de dominio y enums
│   ├── registro/               # Refugio, ABB, árbol de decisión, cédula
│   ├── recomendacion/          # Motor, Estrategia y RecomendadorScoring
│   ├── prioridad/              # GestorPrioridad
│   ├── historial/              # HistorialAdopciones y reportes
│   ├── visitas/                # MapaSectores y PlanificadorVisitas (grafo)
│   └── persistencia/           # lectura/escritura CSV
└── test/ec/udla/refugio/       # casos de prueba por módulo
```

El repositorio está publicado en https://github.com/zaid290/refugio-matching (cuenta `zaid290`) y los demás integrantes (`Esteban201020` y `MaheP-06`) participan como colaboradores. Trabajaremos con una rama `main` protegida, una rama `develop` de integración y ramas `feature/<modulo>` por integrante; cada cambio entra mediante pull request revisado por otro miembro del equipo.

Riesgos técnicos y acciones de mitigación

Riesgo	Impacto	Acción de mitigación
Evaluación de temperamento subjetiva o incompleta.	Alto	Ficha de evaluación con escala 1–5 y criterios escritos; un animal sin evaluación no entra al matching.
Pesos del scoring mal calibrados (recomendaciones poco útiles).	Medio	Pesos configurables; batería de casos de prueba con resultados esperados definidos por el equipo; ajuste en el Progreso 2.
La priorización empuja animales poco compatibles.	Alto	Umbral mínimo de afinidad (0,50) antes de aplicar el bono de prioridad; caso de prueba específico.
Integración tardía entre módulos.	Alto	Definir primero las interfaces y clases del modelo (semana 1); integración semanal en `develop`.
Inconsistencia de estados (dos solicitudes aprobadas para el mismo animal).	Medio	Transiciones de estado centralizadas en SolicitudAdopcion y pruebas de reglas de negocio.

Riesgo	Impacto	Acción de mitigación
Falta de datos reales.	Medio	Generador de datos sintéticos con distribuciones plausibles de edad, especie y temperamento.
Mapa de sectores incompleto o con sectores sin conexión.	Medio	Verificar la conectividad con un recorrido en anchura al cargar el mapa.
Carga de trabajo alta para un equipo de tres integrantes.	Medio	Agrupar los módulos por afinidad (registro; motor y prioridad; historial y visitas) y revisar el avance cada semana.
Alcance excesivo para el plazo.	Medio	Priorizar un producto mínimo con interfaz de consola y seguir el calendario por progresos.

Declaración de uso de herramientas de inteligencia artificial

Herramienta	Uso realizado	Resultado incorporado	Forma de verificación
Claude (Anthropic)	Orientación y apoyo para elaborar el borrador del informe, generar el código del diagrama UML de clases y proponer la organización del repositorio. También se utilizó para resolver dudas y comprender cómo organizar los diferentes componentes del proyecto.	Las propuestas sirvieron como borrador inicial del informe, del diagrama UML y de la estructura del repositorio. El equipo definió la idea, los módulos y las alternativas del proyecto, pidió ajustar el diseño a los temas del sílabo y revisó y adaptó el contenido. La configuración del repositorio en GitHub y el desarrollo del código están a cargo de los integrantes del equipo.	El equipo revisó y verificó el trabajo realizado, comprobando el funcionamiento del código, la coherencia de la documentación, la estructura del repositorio y la consistencia del diagrama UML.

Referencias

Asamblea Nacional del Ecuador. (2021). *Ley Orgánica de Protección de Datos Personales*. Registro Oficial Suplemento 459, 26 de mayo de 2021.

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

Bradley, J., & Rajendran, S. (2021). Increasing adoption rates at animal shelters: A two-phase approach to predict length of stay and optimal shelter allocation. *BMC Veterinary Research, 17*, Artículo 70. https://doi.org/10.1186/s12917-020-02728-2

Brown, W. P., Davidson, J. P., & Zuefle, M. E. (2013). Effects of phenotypic characteristics on the length of stay of dogs at two no kill animal shelters. *Journal of Applied Animal Welfare Science, 16*(1), 2–18. https://doi.org/10.1080/10888705.2013.740967

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Hawes, S. M., Kerrigan, J. M., Hupe, T., & Morris, K. N. (2020). Factors informing the return of adopted dogs and cats to an animal shelter. *Animals, 10*(9), Artículo 1573. https://doi.org/10.3390/ani10091573

Oracle. (s. f.). *Class PriorityQueue<E> (Java SE 24 & JDK 24)*. Recuperado el 18 de octubre de 2026, de https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/PriorityQueue.html

Powell, L., Reinhard, C., Satriale, D., Morris, M., Serpell, J., & Watson, B. (2021). Characterizing unsuccessful animal adoptions: Age and breed predict the likelihood of return, reasons for return and post-return outcomes. *Scientific Reports, 11*, Artículo 8018. https://doi.org/10.1038/s41598-021-87649-2